



TI3214-02-2026-3-PROGRAMACIÓN II

Estudiante:

Víctor Aníbal Olmo Sánchez

Matrícula:

25-1007

Sección:

02

Profesor:

Joerlyn Mariano Morfe Ureña

Tema:

Diseño de Arquitectura de una Aplicación React

Fecha de entrega:

24/06/2026

Escenario seleccionado

Se eligió el escenario de un **Sistema de Gestión de Estudiantes**, una aplicación web pensada para que el personal administrativo de una institución educativa gestione la información de estudiantes, cursos, matrículas y calificaciones. Este escenario permite aplicar de forma clara los conceptos de páginas, componentes reutilizables, manejo de estado y consumo de servicios mediante una API REST, ya que involucra operaciones típicas de un sistema CRUD (crear, leer, actualizar y eliminar información).

Parte 1. Diseño Arquitectónico

A continuación, se presenta el diagrama de arquitectura general de la aplicación. El diagrama muestra cómo el navegador del usuario interactúa con la aplicación React a través de **App.jsx** y el enrutador; cómo las **páginas** se componen de **componentes reutilizables**; cómo el estado se gestiona mediante **Context API** y **hooks personalizados**; y cómo los **servicios** consumen la API REST del backend para obtener y enviar información de estudiantes, cursos y calificaciones. También se indica cómo los recursos estáticos (imágenes y estilos) se utilizan dentro de las páginas y componentes.

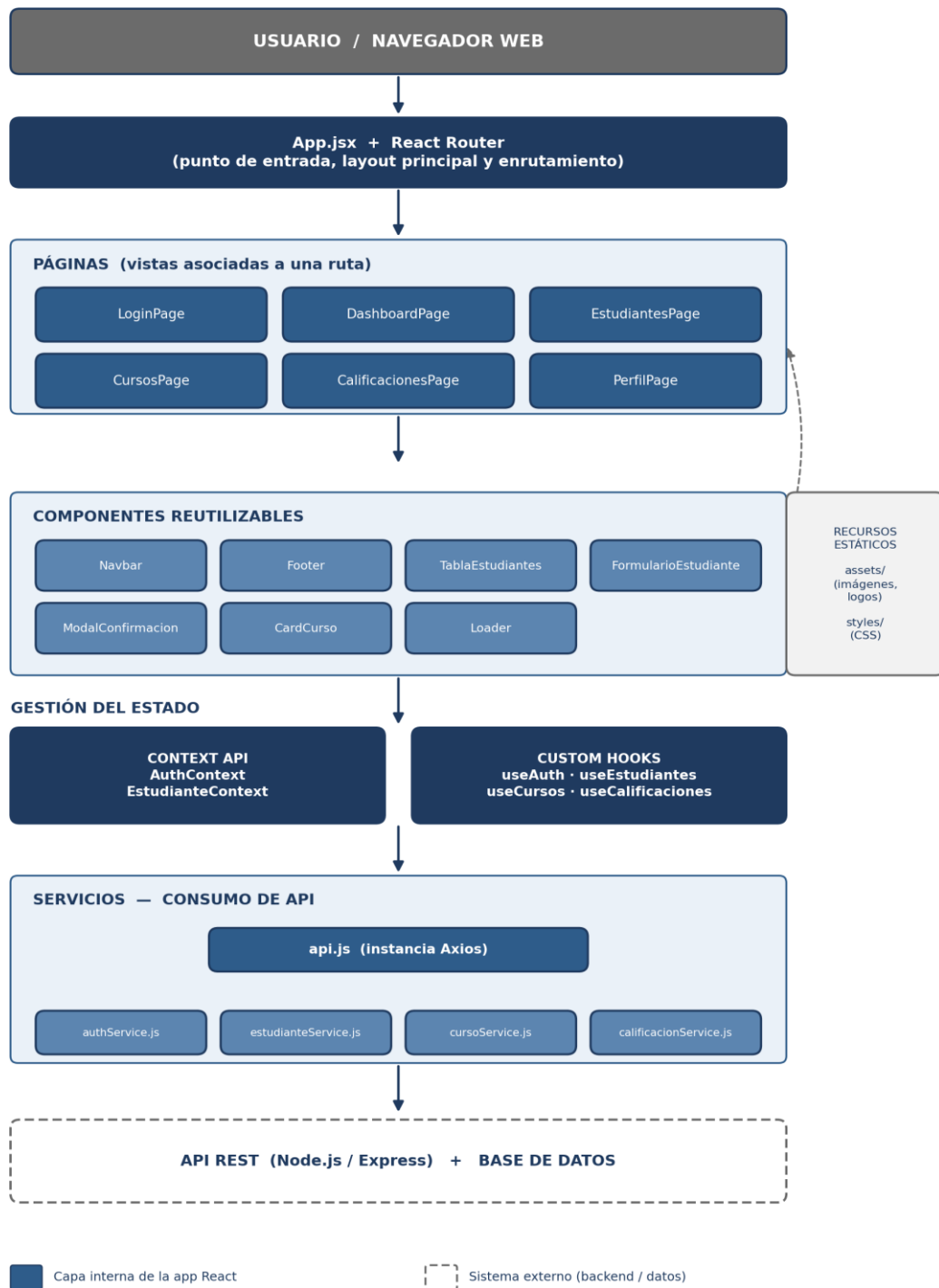


Figura 1. Arquitectura general de la aplicación React para el Sistema de Gestión de Estudiantes.

Parte 2. Estructura del Proyecto

La siguiente estructura de carpetas organiza el proyecto React siguiendo el principio de separación de responsabilidades, de modo que cada tipo de archivo tiene un lugar claramente definido:

```
src/  
|-- components/  
|-- pages/  
|-- services/  
|-- hooks/  
|-- context/  
|-- assets/  
|-- styles/  
|-- routes/  
|-- utils/  
+-- App.jsx
```

Función de cada carpeta

components/	Componentes de interfaz reutilizables que no dependen de una ruta específica: Navbar, Footer, TablaEstudiantes, FormularioEstudiante, ModalConfirmacion, CardCurso y Loader.
pages/	Vistas completas asociadas a una ruta de la aplicación, como LoginPage, DashboardPage, EstudiantesPage, CursosPage, CalificacionesPage y PerfilPage. Cada página combina varios componentes.
services/	Funciones encargadas de la comunicación con la API REST: api.js define la instancia de Axios, y cada archivo (estudianteService.js, cursoService.js, authService.js, calificacionService.js) agrupa las peticiones de una entidad.
hooks/	Hooks personalizados que encapsulan lógica reutilizable de estado y datos, como useAuth, useEstudiantes, useCursos y useCalificaciones, evitando repetir código entre páginas.
context/	Proveedores de Context API para el estado global de la aplicación, como AuthContext (sesión del usuario) y EstudianteContext (datos compartidos de estudiantes).
assets/	Recursos estáticos como imágenes, logotipos e iconos utilizados en la interfaz.
styles/	Hojas de estilo globales, variables de color y configuración de estilos compartidos por toda la aplicación.
routes/	Definición centralizada de las rutas de la aplicación y componentes de protección de rutas privadas (por ejemplo, PrivateRoute para restringir el acceso al panel).

utils/	Funciones auxiliares de uso general: formateo de fechas, cálculo de promedios de calificaciones y validaciones de formularios.
App.jsx	Componente raíz que monta los proveedores de contexto y el enrutador principal de la aplicación.

Parte 3. Identificación de Componentes

Se identifican siete componentes reutilizables que existirán dentro de la aplicación, junto con su función principal y las propiedades (props) que recibirán:

Componente	Función principal	Props
Navbar	Muestra el menú de navegación superior con acceso a las secciones principales del sistema y la opción de cerrar sesión.	usuario, onLogout
Footer	Muestra información institucional y enlaces de contacto en la parte inferior de cada página.	anio (opcional)
TablaEstudiantes	Renderiza el listado de estudiantes en formato de tabla, con acciones para editar o eliminar cada registro.	estudiantes, onEditar, onEliminar
FormularioEstudiante	Formulario reutilizable que permite crear un nuevo estudiante o editar uno existente.	estudianteInicial, onGuardar, onCancel
ModalConfirmacion	Ventana modal genérica para confirmar acciones críticas, como eliminar un estudiante o un curso.	mensaje, visible, onConfirmar, onCancel
CardCurso	Tarjeta visual que resume la información de un curso: nombre, docente a cargo y cupos disponibles.	curso, onSeleccionar
Loader	Indicador de carga reutilizable que se muestra mientras se obtienen datos desde la API.	mensaje (opcional)

Parte 4. Reflexión

¿Por qué es importante diseñar la arquitectura antes de iniciar el desarrollo?

Diseñar la arquitectura antes de programar permite anticipar cómo se relacionarán los distintos módulos de la aplicación, evitando improvisaciones que generan código desordenado y difícil de escalar. Al planear desde el inicio la estructura de carpetas, los componentes y el flujo de datos, el equipo de desarrollo reduce el riesgo de tener que reescribir partes completas del sistema cuando el proyecto crece. Además, una arquitectura definida sirve como guía común para todos los integrantes, lo que facilita dividir tareas e incorporar nuevos desarrolladores sin perder la coherencia del proyecto.

¿Qué ventajas ofrece React al trabajar con componentes reutilizables?

React permite encapsular la interfaz y la lógica de una parte específica de la aplicación dentro de un componente que puede reutilizarse en distintas pantallas, como ocurre con `TablaEstudiantes` o `ModalConfirmacion` en este proyecto. Esto reduce la duplicación de código, ya que un mismo componente recibe diferentes props según el contexto en que se use. También facilita el mantenimiento, porque al corregir un error o mejorar el diseño de un componente, el cambio se refleja automáticamente en todos los lugares donde se utiliza. Por último, los componentes reutilizables hacen que el código sea más legible y más fácil de probar de forma aislada.

¿Cómo influye una buena arquitectura en el mantenimiento de una aplicación?

Una arquitectura bien organizada, con responsabilidades claramente separadas entre páginas, componentes, servicios y manejo de estado, hace que cualquier modificación futura sea más sencilla porque se sabe exactamente en qué carpeta o archivo se debe trabajar. Por ejemplo, si cambia la forma en que se consume la API de estudiantes, solo es necesario modificar `estudianteService.js` sin afectar la interfaz de las páginas. Esto reduce el riesgo de introducir errores al actualizar el sistema, acelera la incorporación de nuevas funcionalidades y permite que distintos miembros del equipo trabajen en paralelo sobre diferentes capas de la aplicación sin generar conflictos.